Function_Assignment_By_ChinamyeeNayak

In [1]:
```
#Q1: Define a function calls addNumber(x, y) that takes in two number and returns t
```

In [2]:
```
# Solution
def addNumber(x, y):
    return x + y

# Example usage:
result = addNumber(5, 3)
print(result)  # Output: 8
```

8

In [3]:
```
#Q2: Define a function calls subtractNumber(x, y) that takes in two numbers and ret
```

In [4]:
```
# Solution
def subtractNumber(x, y):
    return x - y

# Example usage:
result = subtractNumber(8, 3)
print(result)  # Output: 5
```

5

In [5]:
```
#Q3: Write a function getBiggerNumber(x, y) that takes in two numbers as arguments
```

In [6]:
```
# Solution
def getBiggerNumber(x, y):
    return max(x, y)

# Example usage:
result = getBiggerNumber(5, 8)
print(result)  # Output: 8
```

8

In [7]:
```
# Q4: Python provides many built-in modules with many useful functions.
# One such module is the math module. The math module provides many useful function
# You will need to do a "import math" before you are allowed to use the functions w


# Calculate the square root of 16 and stores it in the variable a


# Calculate 3 to the power of 5 and stores it in the variable b


# Calculate area of circle with radius = 3.0 by making use of the math.pi constant
```

In [8]:
```python
# Solution
import math

# Calculate the square root of 16
a = math.sqrt(16)

# Calculate 3 to the power of 5
b = math.pow(3, 5)

# Calculate the area of a circle with a radius of 3.0
c = math.pi * math.pow(3.0, 2)

# Print the results
print("Square root of 16:", a)
print("3 to the power of 5:", b)
print("Area of a circle with radius 3.0:", c)
```

```
Square root of 16: 4.0
3 to the power of 5: 243.0
Area of a circle with radius 3.0: 28.274333882308138
```

In [9]:
```python
#Q5: Write a function to convert temperature from Celsius to Fahrenheit scale.
# oC to oF Conversion: Multipy by 9, then divide by 5, then add 32.

# Note: Return a string of 2 decimal places.
# In - Cel2Fah(28.0)
# Out - '82.40'
# In - Cel2Fah(0.00)
# Out - '32.00'
```

In [10]:
```python
# Solution

def Cel2Fah(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return '{:.2f}'.format(fahrenheit)

# Example usage:
result1 = Cel2Fah(28.0)
print(result1)  # Output: '82.40'

result2 = Cel2Fah(0.00)
print(result2)  # Output: '32.00'
```

```
82.40
32.00
```

In [11]:
```python
#Q6: Write a function to compute the BMI of a person.
#     BMI = weight(kg)  /  ( height(m)*height(m) )

# Note: Return a string of 1 decimal place.
# In - BMI(63, 1.7)
# Out - '21.8'
# In - BMI(110, 2)
# Out - '27.5'
```

In [12]:
```
# Solution
def BMI(weight, height):
    bmi = weight / (height * height)
    return '{:.1f}'.format(bmi)

# Example usage:
result1 = BMI(63, 1.7)
print(result1)  # Output: '21.8'

result2 = BMI(110, 2)
print(result2)  # Output: '27.5'
```

```
21.8
27.5
```

In [14]:
```
#Q7: Write a function percent(value, total) that takes in two numbers as arguments,
# In - percent(46, 90)
# Out - 51
# In - percent(51, 51)
# Out - 100
# In - percent(63, 12)
# Out - 525
```

In [15]:
```
# Solution

def percent(value, total):
    percentage = (value / total) * 100
    return int(percentage)

# Example usage:
result1 = percent(46, 90)
print(result1)  # Output: 51

result2 = percent(51, 51)
print(result2)  # Output: 100

result3 = percent(63, 12)
print(result3)  # Output: 525
```

```
51
100
525
```

In [16]:
```
#Q8: Write a function to compute the hypotenuse given sides a and b of the triangle
# Hint: You can use math.sqrt(x) to compute the square root of x.
# In - hypotenuse(3, 4)
# Out - 5
# In - hypotenuse(5, 12)
# Out - 13
```

In [17]:
```
# Solution

import math

def hypotenuse(a, b):
    return int(math.sqrt(a**2 + b**2))
```

```
# Example usage:
result1 = hypotenuse(3, 4)
print(result1)  # Output: 5

result2 = hypotenuse(5, 12)
print(result2)  # Output: 13
```

```
5
13
```

In [18]:
```
#Q9: Write a function getSumOfLastDigits() that takes in a list of positive numbers
# getSumOfLastDigits([2, 3, 4])
# 9
# getSumOfLastDigits([1, 23, 456])
# 10
```

In [19]:
```
# solution
def getSumOfLastDigits(numbers):
    last_digits = [num % 10 for num in numbers]
    return sum(last_digits)

# Example usage:
result1 = getSumOfLastDigits([2, 3, 4])
print(result1)  # Output: 9

result2 = getSumOfLastDigits([1, 23, 456])
print(result2)  # Output: 10
```

```
9
10
```

In [20]:
```
#Q10: Write a function that uses a default value.
# In - introduce('Lim', 20)
# Out - 'My name is Lim. I am 20 years old.'
# In - introduce('Ahmad')
# Out - 'My name is Ahmad. My age is secret.'
```

In [21]:
```
# Solution
def introduce(name, age="secret"):
    return f"My name is {name}. My age is {age}."

# Example usage:
result1 = introduce('Lim', 20)
print(result1)  # Output: 'My name is Lim. I am 20 years old.'

result2 = introduce('Ahmad')
print(result2)  # Output: 'My name is Ahmad. My age is secret.'
```

```
My name is Lim. My age is 20.
My name is Ahmad. My age is secret.
```

In [22]:
```
#Q11: Write a function isEquilateral(x, y, z) that accepts the 3 sides of a triangl
# The program should return True if it is an equilateral triangle.

# In - isEquilateral(2, 4, 3)
# False - False
```

```
# In - isEquilateral(3, 3, 3)
# Out - True
# In - isEquilateral(-3, -3, -3)
# Out - False
```

In [23]:
```
# solution
def isEquilateral(x, y, z):
    if x <= 0 or y <= 0 or z <= 0:
        return False  # Negative or zero side lengths cannot form a triangle.
    return x == y == z

# Example usage:
result1 = isEquilateral(2, 4, 3)
print(result1)  # Output: False

result2 = isEquilateral(3, 3, 3)
print(result2)  # Output: True

result3 = isEquilateral(-3, -3, -3)
print(result3)  # Output: False
```

```
False
True
False
```

In [24]:
```
#Q12: For a quadratic equation in the form of ax2+bx+c, the discriminant, D is b2-4
# In - quadratic(1, 2, 3)
# Out - 'The discriminant is -8.'
# In - quadratic(1, 3, 2)
# Out - 'The discriminant is 1.'
# In - quadratic(1, 4, 4)
# Out - 'The discriminant is 0.'
```

In [25]:
```
# solution

def quadratic(a, b, c):
    discriminant = b**2 - 4*a*c
    return f'The discriminant is {discriminant}.'

# Example usage:
result1 = quadratic(1, 2, 3)
print(result1)  # Output: 'The discriminant is -8.'

result2 = quadratic(1, 3, 2)
print(result2)  # Output: 'The discriminant is 1.'

result3 = quadratic(1, 4, 4)
print(result3)  # Output: 'The discriminant is 0.'
```

```
The discriminant is -8.
The discriminant is 1.
The discriminant is 0.
```

In [26]:
```
#Q13: Define a function calls addFirstAndLast(x) that takes in a list of numbers an
# In - addFirstAndLast([])
# Out - 0
```

```
# In - addFirstAndLast([2, 7, 3])
# Out - 5
# In - addFirstAndLast([10])
# Out - 10

def addFirstAndLast(lst):
    pass

addFirstAndLast([2,7,3])
addFirstAndLast([])
addFirstAndLast([10])
```

In [27]:
```
# Solution

def addFirstAndLast(lst):
    if len(lst) < 1:
        return 0
    elif len(lst) == 1:
        return lst[0]
    else:
        return lst[0] + lst[-1]

# Example usage:
result1 = addFirstAndLast([])
print(result1)  # Output: 0

result2 = addFirstAndLast([2, 7, 3])
print(result2)  # Output: 5

result3 = addFirstAndLast([10])
print(result3)  # Output: 10
```

```
0
5
10
```

#Q14: Complete the 'lambda' expression so that it returns True if the argument is an even number, and False otherwise. (lambda num : ----------)(6) # Output : True

In [29]:
```
# Solution
(lambda num: num % 2 == 0)(6)
```

Out[29]:
```
True
```

In [30]:
```
#Q15: Get the documentation of below function

def getScore():
  ''''A function that computes and returns the final score.'''
  pass
#Output: "'A function that computes and returns the final score."
```

In [31]:
```
# Solution
print(getScore.__doc__)
#The __doc__ attribute contains the docstring of the function, which is the documen
#'A function that computes and returns the final score.'
```

```
'A function that computes and returns the final score.
```

In [32]:
```
#Q16: In Python, it is possible to pass a function as a argument to another functio
# Write a function useFunction(func, num) that takes in a function and a number as
# The useFunction should produce the output shown in the examples given below.

# def addOne(x):
#   return x + 1
# useFunction(addOne, 4)
# 25
# useFunction(addOne, 9)
# 100
# useFunction(addOne, 0)
# 1
```

In [33]:
```
# Solution

def useFunction(func, num):
    result = func(num)
    return result * result

def addOne(x):
    return x + 1

# Example usage:
result1 = useFunction(addOne, 4)
print(result1)  # Output: 25

result2 = useFunction(addOne, 9)
print(result2)  # Output: 100

result3 = useFunction(addOne, 0)
print(result3)  # Output: 1
```

```
25
100
1
```

In [34]:
```
# Q17: Write a function find_max that accepts three numbers as arguments and return
# Write another function main, in main() function accept three numbers from user an
```

In [35]:
```
# Solution
def find_max(a, b, c):
    return max(a, b, c)

def main():
    num1 = float(input("Enter the first number: "))
    num2 = float(input("Enter the second number: "))
    num3 = float(input("Enter the third number: "))

    largest = find_max(num1, num2, num3)
    print(f"The largest number among {num1}, {num2}, and {num3} is {largest}.")

if __name__ == "__main__":
    main()
```

```
Enter the first number: 23
Enter the second number: 33
Enter the third number: 11
The largest number among 23.0, 33.0, and 11.0 is 33.0.
```

In [36]:
```
#Q18: Write a function, is_vowel that returns the value true if a given character i
# and otherwise returns false.
# Write another function main, in main() function accept a string from user and cou
# of vowels in that string.
```

In [37]:
```
# Solution
def is_vowel(char):
    vowels = "aeiouAEIOU"
    return char in vowels

def main():
    input_string = input("Enter a string: ")
    count = 0
    for char in input_string:
        if is_vowel(char):
            count += 1
    print(f"The number of vowels in the string is: {count}")

if __name__ == "__main__":
    main()
```

```
Enter a string: chinmayee
The number of vowels in the string is: 4
```

In [38]:
```
#Q19: Write a function named is_prime, which takes an integer as an argument and re
# if the argument is a prime number, or false otherwise.
# Also, write the main function that displays prime numbers between 1 to 500.
```

In [39]:
```
# Solution

def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def main():
    print("Prime numbers between 1 to 500:")
    for num in range(1, 501):
        if is_prime(num):
            print(num, end=", ")

if __name__ == "__main__":
    main()
```

```
Prime numbers between 1 to 500:
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 17
3, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 2
69, 271, 277, 281, 283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367,
373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463,
467, 479, 487, 491, 499,
```

In [40]:
```
#Q20: Write a function in python to find the sum of the cube of elements in a list.
# The list is received as an argument to the function, in turn, the function must r
# Write the main function which invokes the above function.
```

In [41]:
```
# Solution

def sum_of_cubes(lst):
    cube_sum = 0
    for num in lst:
        cube_sum += num**3
    return cube_sum

def main():
    input_list = [1, 2, 3, 4, 5]  # Replace with your list
    result = sum_of_cubes(input_list)
    print(f"The sum of the cubes of elements in the list is: {result}")

if __name__ == "__main__":
    main()
```

```
The sum of the cubes of elements in the list is: 225
```

In [42]:
```
#Q21: Write the definition of a function zero_ending(scores) to add all those value
# which are ending with zero and display the sum.
# For example: If the scores contain [200, 456, 300, 100, 234, 678] The sum should
```

In [44]:
```
# Solution
def zero_ending(scores):
    total = 0
    for score in scores:
        if score % 10 == 0:
            total += score
    return total

scores = [200, 456, 300, 100, 234, 678]
result = zero_ending(scores)
print(f"The sum of scores ending with zero is: {result}")
```

```
The sum of scores ending with zero is: 600
```

In [45]:
```
#Q22: Write a definition of a method count_now(places) to find and display those pl
# in which there are more than 5 characters.

# For example :
# If the list places contains
# ["DELHI","LONDON","PARIS","NEW YORK","DUBAI"]
# The following should get displayed :
```

```
# LONDON
# NEW YORK
```

In [46]:
```
# Solution

def count_now(places):
    for place in places:
        if len(place) > 5:
            print(place)

places = ["DELHI", "LONDON", "PARIS", "NEW YORK", "DUBAI"]
count_now(places)
```

```
LONDON
NEW YORK
```

In [47]:
```
#Q23: Write a method in python to display the elements of list thrice if it is a nu
# the element terminated with ‘#’ if it is not a number.

# For example, if the content of list is as follows :
# ThisList=[‘41’,‘DROND’,‘GIRIRAJ’, ‘13’,‘ZARA’]
# The output should be
# 414141
# DROND#
# GIRIRAJ#
# 131313
# ZARA#
```

In [48]:
```
# Solution

def process_list(this_list):
    for element in this_list:
        if element.isdigit():
            print(element * 3)
        else:
            print(element + '#')

ThisList = ['41', 'DROND', 'GIRIRAJ', '13', 'ZARA']
process_list(ThisList)
```

```
414141
DROND#
GIRIRAJ#
131313
ZARA#
```

In [49]:
```
#Q24: For a given list of values in descending order, write a method in python to s
# for a value with the help of Binary Search method.
# The method should return position of the value and should return -1 if the value
```

In [50]:
```
# Solution
def binary_search_descending(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            right = mid - 1
        else:
            left = mid + 1

    return -1

# Example usage:
my_list = [20, 15, 10, 8, 5, 2]
value_to_find = 10
result = binary_search_descending(my_list, value_to_find)

if result != -1:
    print(f"Value {value_to_find} found at position {result}.")
else:
    print(f"Value {value_to_find} not found in the list.")
```

```
Value 10 found at position 2.
```

In [52]:

```
#Q25: Write a function half_and_half that takes in a list and change the list
# such that the elements of the second half are now in the first half.

# For example, if the size of list is even and content of list is as follows :
# my_liist = [10,20,30,40,50,60]
# The output should be
# [40,50,60,10,20,30]
# if the size of list is odd and content of list is as follows :
# my_liist = [10,20,30,40,50,60,70]
# The output should be
# [50,60,70,40,10,20,30]
```

In [53]:

```
# Solution

def half_and_half(my_list):
    length = len(my_list)
    middle = length // 2

    if length % 2 == 0:  # List size is even
        my_list[:middle], my_list[middle:] = my_list[middle:], my_list[:middle]
    else:  # List size is odd
        my_list[:middle], my_list[middle+1:] = my_list[middle+1:], my_list[:middle]

# Example usage:
my_list1 = [10, 20, 30, 40, 50, 60]
half_and_half(my_list1)
print(my_list1)

my_list2 = [10, 20, 30, 40, 50, 60, 70]
half_and_half(my_list2)
print(my_list2)
```

```
[40, 50, 60, 10, 20, 30]
[50, 60, 70, 40, 10, 20, 30]
```

In [54]:
```
#Q26: Write a function that accepts a dictionary as an argument. If the dictionary
# return an empty dictionary, otherwise, return a new dictionary whose values are n
# keys are the values.
```

In [55]:
```
# Solution

def swap_keys_and_values(input_dict):
    # Check for duplicate values in the input dictionary
    values = list(input_dict.values())
    if len(values) != len(set(values)):
        return {}  # Dictionary contains duplicate values

    # Create a new dictionary with keys and values swapped
    swapped_dict = {value: key for key, value in input_dict.items()}
    return swapped_dict

# Example usage:
my_dict1 = {'a': 1, 'b': 2, 'c': 3}
result1 = swap_keys_and_values(my_dict1)
print(result1)  # Output: {1: 'a', 2: 'b', 3: 'c'}

my_dict2 = {'a': 1, 'b': 2, 'c': 2}
result2 = swap_keys_and_values(my_dict2)
print(result2)  # Output: {}
```

```
{1: 'a', 2: 'b', 3: 'c'}
{}
```

OR

In [56]:
```
def swap_keys_and_values(input_dict):
    # Create a set to store unique values
    unique_values = set()

    # Initialize an empty dictionary to store the swapped keys and values
    swapped_dict = {}

    for key, value in input_dict.items():
        if value in unique_values:
            # Duplicate value found, return an empty dictionary
            return {}
        else:
            # Add the value to the set of unique values and swap the key-value pair
            unique_values.add(value)
            swapped_dict[value] = key

    return swapped_dict

# Example usage:
my_dict1 = {'a': 1, 'b': 2, 'c': 3}
result1 = swap_keys_and_values(my_dict1)
print(result1)  # Output: {1: 'a', 2: 'b', 3: 'c'}
```

```python
my_dict2 = {'a': 1, 'b': 2, 'c': 2}
result2 = swap_keys_and_values(my_dict2)
print(result2)  # Output: {}
```

```
{1: 'a', 2: 'b', 3: 'c'}
{}
```

In [ ]: